

Week 1

1. Базовые понятия RL

Отличие от обучения с учителем. В supervised learning есть пары (x, y) и мы минимизируем потери. Но во многих задачах (реклама, управление роботом) правильных ответов нет - RL решает именно их: эталонов нет, но есть обратная связь на действия агента.

Метод проб и ошибок: инициализировать наивным решением $\rightarrow$ собирать данные $\rightarrow$ научиться сопоставлять (ситуация) $\rightarrow$ (оптимальное действие) $\rightarrow$ повторить.

Bandit - простейшая постановка: наблюдение $\rightarrow$ действие $\rightarrow$ обратная связь (признаки пользователя $\rightarrow$ показать баннер $\rightarrow$ клик). **Decision process** - агент циклически взаимодействует со средой, его действия меняют будущие наблюдения.

Мотивация RL:

1. **Что значит «оптимально»** (прибыль сейчас vs. лояльность)
2. **Исследование** - следуя только текущей стратегии, можно не открыть лучшую.

Формализм MDP: состояния $s \in S$, действия $a \in A$, награды $r \in \mathbb{R}$, динамика $P(s_{t+1}|s_t, a_t)$. Политика $\pi(a|s)$ - вероятность взять a в состоянии s . Марковское предположение - будущее зависит только от текущего состояния и действия:

$$P(s_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}) = P(s_{t+1}|s_t, a_t)$$

Суммарная награда за сессию и цель RL - максимум ожидаемой награды:

$$R = \sum_t r_t, \pi(a|s): E_{\pi}[R] \rightarrow \max$$

Главное отличие RL от обучения с учителем и без учителя: агент сам формирует данные, на которых учится, и его действия влияют на то, что он увидит дальше.

2. Метод кросс-энтропии (табличный + нейронная сеть)

Идея: сэмплировать N сессий $\rightarrow$ выбрать M лучших («элитных») $\rightarrow$ изменить политику так, чтобы она чаще повторяла действия из элитных сессий $\rightarrow$ повторить.

Табличный вариант (Tabular CEM)

Политика - матрица $\pi(a|s)$. Пересчитываем её как частоту действий в элитных играх:

$$\pi(a|s) = \frac{\sum_{(s_t, a_t) \in Elite} [s_t = s][a_t = a]}{\sum_{(s_t, a_t) \in Elite} [s_t = s]}$$

Табличный подход разваливается при больших/бесконечных пространствах состояний (игра по пикселям) - частоты не накопить.

Аппроксимированный вариант (нейросеть)

Политику $\pi_w(a|s)$ аппроксимируют моделью. Сеть учится максимизировать правдоподобие действий из элитных игр - обычная классификация (вход - состояние, класс - действие); шаг градиентного подъёма:

$$W_{i+1} = W_i + \alpha \nabla \sum_{(s_i, a_i) \in Elite} \log \pi_{W_i}(a_i|s_i)$$

Непрерывные действия: вместо классификатора берут регрессию или нормальное распределение $\pi_w(a|s) = N(\mu(s), \sigma^2)$ (MLPRegressor вместо MLPClassifier). **Трюки (Tricks):** помнить сессии за 3–5 итераций; энтропийная регуляризация; RNN для частично наблюдаемых сред.

Week 2

1. V(s), Q(s, a) - что это такое?

Это две **функции ценности**, показывающие, насколько «хорошо» находиться в состоянии (или совершить действие), если дальше следовать политике π . Обе - через ожидаемый возврат G_t .

State-value $V(s)$ - ожидаемый возврат из состояния s при следовании π (уравнение Беллмана):

$$v_{\pi}(s) = E_{\pi}[G_t | S_t = s] = \sum_a \pi(a|s) \sum_{r, s'} p(r, s' | s, a) [r + \gamma v_{\pi}(s')]$$

Action-value $Q(s, a)$ - то же, но в s зафиксировано действие a . Отличие от V : на первом шаге нет стохастичности политики (действие задано):

$$q_{\pi}(s, a) = E_{\pi}[G_t | S_t = s, A_t = a] = \sum_{r, s'} p(r, s' | s, a) [r + \gamma v_{\pi}(s')]$$

Связь: V - усреднение Q по действиям с весами политики; зная оптимальную Q^* , политику легко восстановить **без** модели среды:

$$v_{\pi}(s) = \sum_a \pi(a|s) q_{\pi}(s, a); \pi_*(s) = \operatorname{argmax}_a q_*(s, a)$$

А из V^* напрямую нельзя - для перехода к действиям нужны вероятности переходов $p(r, s' | s, a)$.

2. Метод Policy Iteration

Поиск оптимальной политики чередованием оценки текущей политики и её улучшения (частный случай Generalized Policy Iteration).

Шаг 1. Policy Evaluation. Для фиксированной π итеративно решаем уравнения Беллмана до сходимости ($\Delta < \theta$):

$$V(s) \leftarrow \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$$

Шаг 2. Policy Improvement. Зная v_{π} , строим новую политику жадно по Q - это гарантированно не худшая политика:

$$\pi'(s) \leftarrow \operatorname{argmax}_a \sum_{r, s'} p(r, s' | s, a) [r + \gamma v_{\pi}(s')]$$

Сходимость: если после улучшения $\pi' = \pi$, она удовлетворяет уравнению оптимальности Беллмана - значит оптимальна. В конечном MDP процесс сходится за конечное число итераций.

Сравнение с Value Iteration. В policy iteration оценка идёт до сходимости; в value iteration - одна итерация, затем сразу улучшение.

1. **Policy iteration** - дороже на итерацию ($O(|A||S|^2 + |S|^3)$), но требует мало итераций.
2. **Value iteration** - дешевле на итерацию ($O(|A||S|^2)$), но требует много итераций.
«Серебряной пули» нет: на практике подбирают, сколько шагов тратить на фазу оценки.

Week 3

1. Q-learning, SARSA

Оба - model-free методы на основе **temporal difference (TD)**: истинного $P(s', r | s, a)$ не знают, учатся из переходов $\langle s, a, r, s' \rangle$, обновляя Q скользящим средним по одному сэмплу:

$$Q(s_t, a_t) \leftarrow \alpha \hat{Q}(s_t, a_t) + (1 - \alpha) Q(s_t, a_t)$$

Различие - только в том, как считается улучшенная оценка $\hat{Q}(s, a)$.

Q-learning берёт максимум по следующим действиям - оценивает *оптимальную* политику независимо от того, чем собраны данные → **off-policy**:

$$\hat{Q}(s, a) = r(s, a) + \gamma \max_{a'} Q(s', a')$$

SARSA подставляет действие a' , реально выбранное текущей (напр. ϵ -greedy) политикой → оценивает *эту* политику со всей случайностью → **on-policy** (нужен набор $\langle s, a, r, s', a' \rangle$):

$$\hat{Q}(s, a) = r(s, a) + \gamma Q(s', a')$$

Expected Value SARSA - вместо одного a' берёт матожидание по политике (меньше дисперсия, может работать off-policy):

$$\hat{Q}(s, a) = r(s, a) + \gamma E_{a' \sim \pi(a'|s)} Q(s', a')$$

В Cliff World Q-learning учит путь вдоль обрыва, но с ϵ -greedy срывается; SARSA учитывает случайность своей политики и выбирает более безопасный путь.

	Q-learning	SARSA
Цель в $\hat{Q}$	$\max_a Q(s', a')$	$Q(s', a')$ для реального a'
Тип	off-policy	on-policy
Что оценивает	оптимальную π^*	текущую π (с исследованием)
Сэмпл	$\langle s, a, r, s' \rangle$	$\langle s, a, r, s', a' \rangle$

2. Exploration vs Exploitation

Фундаментальный компромисс: использовать накопленные знания vs. искать лучшее. Если всегда брать $\arg\max_a Q(s, a)$ при $Q \approx 0$, агент рискует «застрять» (робот, случайно получивший награду за ползание, не узнает, что ходить выгоднее).

ϵ -greedy - с вероятностью ϵ случайное действие, иначе жадное. Просто и гарантирует, что все действия иногда пробуются.

Softmax (Boltzmann) - вероятность пропорциональна softmax от Q; учитывает, насколько действия хороши. Температура τ регулирует случайность:

$$\pi(a|s) = \text{softmax}\left(\frac{Q(s, a)}{\tau}\right)$$

Исследование во времени: для сходимости ϵ постепенно снижают ($\epsilon \rightarrow 0$ - «greedy in the limit»). В нестационарных средах полностью убирать исследование опасно.

Week 4

1. DQN

DQN - это Q-learning, где таблица Q заменена нейросетью-аппроксиматором $Q(s, a; \theta)$: для больших/непрерывных пространств не хранят таблицу, а учат сеть предсказывать Q. Обучение - минимизация MSE до цели по уравнению оптимальности Беллмана:

$$L(\theta) = (Q(s, a; \theta) - [r + \gamma \max_{a'} Q(s', a')])^2$$

Stop gradient: цель $r + \gamma \max_{a'} Q(s', a')$ сама зависит от θ , но при вычислении градиента её считают константой - иначе оптимизировались бы и предсказание, и цель одновременно. На практике добавляют experience replay и target network.

2. Experience replay

Буфер переходов (s, a, r, s') ; для обучения из него случайно сэмпляют мини-батчи. Зачем:

1. **Декорреляция:** соседние переходы скоррелированы, случайная выборка приближает к i. i. d., для которого и предназначен SGD.
2. **Переиспользование опыта** - выше data efficiency.
3. Агент учится **off-policy** на более старой версии самого себя.

Поэтому replay корректно работает только с **off-policy** методами (Q-learning, CEM, Expected Value SARSA); обычный SARSA сломается. В Prioritized Experience Replay переходы берут с приоритетом (по TD-ошибке): распределение (s, a) исказить можно, но r и s' должны оставаться несмещёнными.

3. Target network

Отдельная «замороженная» копия Q-сети с параметрами θ^- , используемая только для цели:

$$y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$$

Проблема - «движущаяся цель». Если цель считать той же сетью, после каждого шага она смещается и обучение расходится. Фиксируя θ^- , получаем задачу, похожую на регрессию с фиксированными метками. Обновление θ^- :

1. **жёстко** - копирование $\theta^- \leftarrow \theta$ раз в N шагов (классический DQN);
2. **мягко** (Polyak) - $\theta^- \leftarrow \tau \theta + (1 - \tau) \theta^-$ с малым τ .

4. Double DQN

Борется с переоценкой Q-значений. Источник смещения - оператор $\max$ в цели: т.к. $E[\max_a Q] \geq \max_a E[Q]$, при зашумлённых оценках один и тот же набор значений используется и для выбора действия, и для его оценки - цель завышается.

Решение: разделить выбор и оценку. Действие-максимизатор выбирает онлайн-сеть θ , а оценивает target-сеть θ^- :

$$y = r + \gamma Q(s', \arg\max_a Q(s', a'; \theta); \theta^-)$$

Разнесение источников шума убирает положительное смещение - оценки ближе к истинным, политика устойчивее.

Week 6

1. Policy gradient

Вместо изучения value-функции напрямую параметризуем политику $\pi_\theta(a|s)$ и максимизируем ожидаемую награду $J = E[R(s, a)]$. Часто проще: выбрать оптимальное действие легче, чем посчитать Q; к тому же value-методы минимизируют MSE предсказания Q, а не качество политики.

Параметры θ «сидят» внутри π_θ . Используем log-derivative trick ($\pi \nabla \log \pi = \nabla \pi$), после подстановки снова получаем матожидание по политике (его можно оценивать сэмплением):

$$\nabla J = E[\nabla \log \pi_\theta(a|s) \cdot Q(s, a)]$$

Формула почти как градиент правдоподобия в supervised, но взвешенный на $Q(s, a)$: «повышаем вероятность» действий пропорционально тому, насколько они хороши.

2. REINFORCE

Базовый policy gradient - оценка ∇J методом Монте-Карло:

$$\nabla J \approx \frac{1}{N} \sum_i \sum_{s, a \in z_i} \nabla \log \pi_\theta(a|s) \cdot Q(s, a)$$

1. Инициализировать веса θ_0 случайно.
2. В цикле: сэмплировать N сессий под π_θ ; оценить ∇J ; шаг подъёма $\theta_{i+1} \leftarrow \theta_i + \alpha \cdot \nabla J$.

Оценка Q через G. Истинное Q недоступно - берут накопленную дисконтированную награду из самой сессии (за один проход назад):

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots = r_t + \gamma G_{t+1}$$

REINFORCE on-policy. Baselines: из Q можно вычесть baseline $b(s)$, зависящий только от состояния - направление градиента не меняется, а дисперсия может упасть, если $b(s)$ коррелирует с Q (лучший выбор $b(s) = V(s)$):

$$\nabla J = E[\nabla \log \pi_\theta(a|s) (Q(s, a) - b(s))]$$

3. Actor-critic

Учить одновременно политику $\pi_\theta(a|s)$ (**actor**) и ценность $V_\theta(s)$ (**critic**): $V(s)$ служит хорошим baseline. В A2C используют advantage, оцениваемый прямо из перехода:

$$A(s, a) = Q(s, a) - V(s) = r + \gamma V(s') - V(s)$$

Две функции потерь (голова делят общее тело сети); $A(s, a)$ при обновлении actor считается константой:

1. **Actor:** $\nabla J \approx \frac{1}{N} \sum \nabla \log \pi_\theta(a|s) \cdot A(s, a)$
2. **Critic (TD-регрессия):** $L \approx \frac{1}{N} \sum (V_{\theta(s)} - [r + \gamma V(s')])^2$

Непрерывные действия: подставить нормальное распределение $\pi(a|s)$ и сэмплировать из него - преимущество policy-based методов. **Трюки (Tricks):** энтропийная регуляризация; параллельные сессии вместо большого replay; logsoftmax. Масштабируемые версии - Asynchronous advantage actor-critic и IMPALA.

Week 7

1. Применение RNN для генерации последовательностей

Encoder-decoder (seq2seq). Энкодер сжимает вход в скрытое представление («thought vector»), декодер пошагово генерирует токены, подавая предыдущий токен и обновляя состояние h . Так решают перевод, image-to-caption, grapheme-to-phoneme, диалог и др.

Обучение - максимизация $\log P(y|x)$ на парах (вход, эталон), т.е. кросс-энтропия с учителем.

Обучение vs. инференс. Энкодер одинаков; декодер меняется: при обучении на вход подаётся эталонный токен (teacher forcing), при инференсе - то, что модель сгенерировала сама (обычно argmax/greedy).

Главная проблема - distribution shift (exposure bias): распределения скрытых состояний при обучении и генерации различаются; стоит модели выдать что-то новое - ошибки начинают накапливаться. Плюс у задачи часто несколько правильных ответов, и лучшее $\log$ -правдоподобие $\neq$ лучшие генерации.

2. Применение RL к дообучению моделей генерации последовательностей

	Supervised	Reinforcement learning
Что нужно	хороший эталон y_{opt}	функция награды $r(s, a)$
На чём учится	на эталонных сессиях (reference)	на собственном выводе (generated)
Плюсы	просто, малая дисперсия	не нужен эталон, нет distribution shift
Минусы	нужен датасет, distribution shift	cold start, большая дисперсия

Связка SL + RL: supervised - претренин (с нуля, малая дисперсия), RL - посттренин (улучшает, не страдает от distribution shift). RL «с нуля» почти не способен случайно набрести на хороший вывод.

SCST (self-critical sequence training). Baseline берут из самого режима инференса (greedy): награду сэмплированной последовательности сравнивают с наградой жадной генерации (она используется в продакшене и не добавляет дисперсии):

$$A(s, a) = R(s, a) - R(s, a_{interference}(s))$$

Пример - image captioning (MSCOCO): претренин по $\log P(\text{caption}|\text{image})$, затем fine-tuning под целевую метрику (CIDEr) с self-critical baseline даёт лучший результат, чем кросс-энтропия.

RLHF и DPO

RLHF (InstructGPT/ChatGPT) - три стадии:

1. **SFT** (дообучение на «инструкция → ответ»);
2. **reward model** (обучается на ранжировках ответов по парному лоссу);
3. **RL** (модель генерирует ответы, reward model их оценивает, политику обучают через PPO с KL-штрафом за отклонение от π_{ref}).

DPO - упрощение **RLHF** без отдельной reward model и без RL-цикла: из RLHF-цели с KL выводится closed-form оптимальная политика, через неё награда выражается как

$$r(x, y) = \beta \log \frac{\pi_r(y|x)}{\pi_{ref}(y|x)} + \beta \log Z(x)$$

Подстановка в парную модель предпочтений сокращает нормировку $Z(x)$, и обучение сводится к простому лоссу на парах (выигрышный y_w , проигрышный y_l). DPO не хуже PPO и устойчивее к температуре.